WEEK-10

Exception Handling Experiments

1) Write a Java program to demonstrate **exception handling using try and catch** by handling division by zero.

PROGRAM:

```
public class DivisionByZeroDemo {
public static void main(String[] args) {
int numerator = 10;
int denominator = 0;
try {
System.out.println("Attempting division...");
int result = numerator / denominator;
System.out.println("Result: " + result);
}
catch (ArithmeticException e) {
System.out.println("Error Caught: Cannot divide an integer by zero!");
System.out.println("Exception message details: " + e.getMessage());
}
System.out.println("The program continues running successfully.");
}
}
```

2) Write a Java program to demonstrate different **types of exceptions** such as `ArithmeticException`, `ArrayIndexOutOfBoundsException`, and `NullPointerException`.

PROGRAM:

```
public class SimpleExceptionDemo {
public static void main(String[] args) {
try {
int result = 10 / 0;
} catch (ArithmeticException e) {
System.out.println("Error 1: You cannot divide by zero!");
}
try {
int[] numbers = {1, 2, 3};
System.out.println(numbers[5]);
} catch (ArrayIndexOutOfBoundsException e){
System.out.println("Error 2: That array position does not exist!");
}
try {
String text = null;
System.out.println(text.length());
} catch (NullPointerException e) {
System.out.println("Error 3: The variable is empty (null)!");
}
}
}
```

3) Write a Java program to demonstrate an **uncaught exception** and observe the default JVM exception message and stack trace.

PROGRAM:

```
public static void main(String[] args) {
System.out.println("Program starts...");
```

```
computeDivision(10, 0);
System.out.println("Program ends...");
}
public static void computeDivision(int a, int b)
{
int result = a / b;
System.out.println("Result: " + result);
}
}
```

4) Write a Java program to handle **multiple exceptions using multiple catch clauses**.

PROGRAM:

```
public class SimpleException {
public static void main(String[] args) {
try {
int a = 10;
int b = 0;
int result = a / b;
System.out.println("Result: " + result);
}
catch (ArithmeticException e) {
System.out.println("Error: Do not divide by zero.");
} catch (ArrayIndexOutOfBoundsException e) {
System.out.println("Error: Array index is out of bounds.");
}
System.out.println("Program keeps running...");
}
}
```

5) Write a Java program to demonstrate the use of the **throw statement** to explicitly generate an exception when a given condition is violated.

PROGRAME:

```
public class SimpleThrowDemo {
public static void main(String[] args) {
int number = -5;
System.out.println("Checking number...");
if (number < 0) {
throw new IllegalArgumentException("Error: Numbers cannot be negative!");
}
System.out.println("The number is valid: " + number);
}
}
```

6) Write a Java program to demonstrate the use of the **throws keyword** by propagating an exception to the calling method.

PROGRAM:

```
public class SimpleThrowsDemo {
public static void checkAge(int age) throws Exception {
if (age < 18) {
throw new Exception("You are too young to vote!");
}
System.out.println("You are allowed to vote.");
}
public static void main(String[] args) {
try {
checkAge(15);
```

```
}
catch (Exception e) {
 System.out.println("Error caught: " + e.getMessage());
}
}
}
```

7) Write a Java program to demonstrate the use of the **finally block** for executing statements regardless of whether an exception occurs.

PROGRAM:

```
public class SimpleFinallyDemo {
public static void main(String[] args) {
try {
System.out.println("Trying to divide 10 by 0...");
int result = 10 / 0;
}
catch (ArithmeticException e) {
System.out.println("Caught an error: Cannot divide by zero.");
} finally {
System.out.println("Finally: This runs NO MATTER WHAT.");
}
System.out.println("-----------------------------------");
try {
System.out.println("Trying to divide 10 by 2...");
int result = 10 / 2;
}
catch (ArithmeticException e) {
System.out.println("Caught an error.");
```

```
} finally {
System.out.println("Finally: This STILL runs NO MATTER WHAT.");
}
}
}
```